The Ch(e)at Code

Agent Patterns for Copilot Chat

⇑ ⇑ ⇓ ⇓ ⇐ ⇒ ⇐ ⇒ B A START

ISSUE #019 • AUGUST 4, 2026 • 🔧 PRACTICAL BUILD

Last issue, we named the Custom Connector pattern. This issue, we build one. The customer scenario is the same: an internal API that no pre-built connector reaches — a legacy employee directory, a proprietary case management system, a custom compliance engine. The agent can't do its job because it can't reach the data. Today we fix that, end to end.

This issue's build: A complete custom connector pipeline — from OpenAPI spec to Copilot Studio plugin action to Azure API Management gateway — that wraps any REST API and exposes it to your agent. Five components, one afternoon. (For the architectural decisions behind these choices, see **Issue #018**.)

🎮 AGENT SPOTLIGHT

The Custom Connector Build

Copilot Studio + Custom Connectors + Power Automate + Azure API Management

The scenario: A customer has an internal Employee Directory API — a REST service that returns employee profiles, team assignments, and current project data. No pre-built connector exists. The Copilot Studio agent they're building needs to look up employees by name and pull their details into conversational responses. Today, users have to leave Copilot, open the internal portal, search manually, and copy-paste the results. The agent should do this in one turn.

The solution: Wrap the internal API with a custom connector, expose it as a plugin action in Copilot Studio, front it with Azure API Management for production resilience, and add an error handling topic so failures degrade gracefully instead of crashing the conversation.

Why this stack: Copilot Studio provides the conversational layer. The custom connector provides reusable, governed API access across the Power Platform. APIM provides the production-grade gateway (rate limiting, caching, retry). Power Automate handles error logging and ops visibility. Each layer does one thing well.

🧩 **PATTERN BREAKDOWN**

Five Components, Build Order

OpenAPI spec → Power Platform connector → Copilot Studio action → APIM gateway → error handling

Five components, in build order:

1. **Write the OpenAPI spec.** Start from your existing API documentation (or even example curl commands) and use Copilot Chat to generate the OpenAPI 3.0 spec. Minimum required sections: `info`, `servers`, `paths` with one GET and one POST, `components/schemas` for request/response. The critical detail: write `summary` and `description` on every operation for the LLM, not for developers.

OPERATION DESCRIPTION — BEFORE & AFTER

```
✗ "GET /api/employees/{id}"
✓ "Look up an employee's profile, team,
   and current project assignments by
   their employee ID"
```

2. **Create the custom connector in Power Platform.** In the Maker Portal: Custom Connectors → Import from OpenAPI. Configure authentication to match the API's auth model (see Issue #018 for the decision matrix). Test each operation directly in the connector editor before proceeding. Key gotcha: the response schema must match exactly — if the API returns extra fields, they're silently dropped unless declared in the spec.

3. **Expose as a Copilot Studio plugin action.** In Copilot Studio: Actions → Add a connector action. Map input parameters to conversational entities (e.g., "employee name" → `employee_id` lookup). Map output fields to the agent's response template. Add a natural-language description: "Use this action when the user asks about an employee's details, team, or current project." This description is what the LLM reads to decide when to invoke the action.

4. **Azure API Management gateway (production).** For production deployments: front the internal API with Azure APIM. The connector points to APIM, not directly to the backend. This adds rate limiting (protect the backend from agent-generated traffic spikes), caching (reduce latency for repeated lookups), retry policies (handle transient failures automatically), and request/response logging (ops visibility without changing the API).

5. **Error handling topic in Copilot Studio.** When the connector returns an error (timeout, 500, auth failure), don't show the raw error to the user. Build a topic that catches the failure and responds: "I wasn't able to reach the employee directory right now. Here's what I can tell you from cached data, or I can try again in a moment." Power Automate logs the error for ops visibility. Graceful degradation beats cryptic failure every time.

END-TO-END PIPELINE

```
User asks: "What team is Sarah on?"
  ↓
Copilot Studio → LLM reads action description, invokes connector
  ↓
Custom connector → calls APIM endpoint
  ↓
Azure APIM → rate limit → cache check → forward to backend
  ↓
Internal API → returns employee profile
  ↓
Copilot Studio → formats response: "Sarah is on the Platform
  Engineering team, working on Project Atlas."
```

Key insight: The connector is a platform asset, not a one-off integration. Once built, any agent or flow in the Power Platform environment can reuse it. Build one connector for the employee directory, and every team that builds an agent gets employee lookup for free. This is how custom connectors compound — each one you build makes the next agent cheaper to create.

⚠ **Build warning:** Don't skip the APIM layer for production deployments. Without it, your agent can hammer the backend API with unbounded requests

— especially during demos when multiple users hit it simultaneously. APIM's rate limiting and caching protect both the backend and the user experience. For demos, pointing directly at the API is fine. For production, always go through APIM.

🛠️ **TRY THIS NOW**

Wrap Any REST API and Expose It to Your Agent

1. **Find an internal REST API with documentation.** Even a Swagger page works. If the customer has an existing API portal, start there. If not, any REST endpoint with predictable request/response formats will do. The goal is a real API, not a mock — you want to demo with live data.

2. **Use Copilot Chat to generate an OpenAPI spec from the docs.** Paste the API documentation into Copilot Chat and ask: "Generate an OpenAPI 3.0 specification for this API. Include clear, scenario-specific descriptions on every operation." Review and adjust the output — pay special attention to the `description` fields.

3. **Import into Power Platform as a custom connector, configure auth, test.** Maker Portal → Custom Connectors → Import from OpenAPI file. Set up the authentication method that matches your API. Test every operation in the connector editor. Don't proceed until every test call returns expected data.

4. **Add as a plugin action in your Copilot Studio agent with clear operation descriptions.** Map inputs to conversational entities, map outputs to response fields, and add natural-language descriptions that tell the LLM exactly when to invoke this action. Test with real questions: "What team is Sarah on?" — the agent should invoke the connector and return a formatted answer.

💡 **The line that opens doors:** "Your customer's most valuable data lives behind APIs that Copilot can't reach today. One custom connector changes that — and you can build it in an afternoon."

Got an agent or pattern to share?

Reply to this email and you might be featured in a future issue.

The Ch(e)at Code

AIWF Team • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Microsoft Internal • Not for external distribution